

# **LAPORAN TUGAS RETRIEVAL-AUGMENTED GENERATION**



**Mata Kuliah :  
Sistem Temu Kembali A**

**Kelompok 6:**

|                                      |                   |
|--------------------------------------|-------------------|
| <b>Kin Kareem Binami Wijaya</b>      | <b>2305551072</b> |
| <b>Adam Rifki Saputra</b>            | <b>2305551078</b> |
| <b>I Gede Metra Shandy Mahardika</b> | <b>2305551110</b> |

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS UDAYANA**

**2026**

## DAFTAR ISI

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| DAFTAR ISI .....                                                                   | 2  |
| DAFTAR GAMBAR .....                                                                | 3  |
| DAFTAR KODE PROGRAM .....                                                          | 4  |
| BAB I RETRIEVAL-AUGMENTED GENERATION .....                                         | 5  |
| 1.1 Apa Itu Retrieval-Augmentes Generation .....                                   | 5  |
| BAB II METODE PENCARIAN INFORMASI PADA RAG .....                                   | 6  |
| 2.1 Lexical Retrieval.....                                                         | 6  |
| 2.2 Sematic Retrieval.....                                                         | 6  |
| 2.3 Perbedaan Lexical Retrieval dan Sematic Retrieval.....                         | 7  |
| BAB III ARSITEKTUR RAG, MANFAAT, DAN PERBANDINGANNYA DENGAN FINE-TUNING .....      | 9  |
| 3.1 Arsitektur Dasar RAG .....                                                     | 9  |
| 3.2 Manfaat RAG.....                                                               | 9  |
| 3.3 Perbedaan RAG Dengan Fine-Tuning.....                                          | 10 |
| BAB IV PEMBAHASAN DAN UJI COBA .....                                               | 12 |
| 4.1 Uji Coba Program .....                                                         | 12 |
| 4.2 Pembahasan Hasil dan Analisis Uji Coba.....                                    | 16 |
| 4.2.1 Analisis Fase <i>Retrieval</i> (Pencarian Konteks).....                      | 17 |
| 4.2.2 Analisis Fase <i>Generation</i> (Pemrosesan LLM dan Prompt Engineering)..... | 18 |
| BAB V KESIMPULAN.....                                                              | 19 |
| 5.1 Kesimpulan.....                                                                | 19 |
| DAFTAR PUSTAKA.....                                                                | 20 |
| LAMPIRAN.....                                                                      | 21 |

## DAFTAR GAMBAR

|                                  |    |
|----------------------------------|----|
| Gambar 4. 1 Hasil Retrievel..... | 17 |
| Gambar 4. 2 Output Jawaban.....  | 17 |

## DAFTAR KODE PROGRAM

|                                                               |    |
|---------------------------------------------------------------|----|
| Kode Program 4. 1 Inisialisasi API dan ChromeDB.....          | 12 |
| Kode Program 4. 2 Memasukan Document.....                     | 13 |
| Kode Program 4. 3 Proses Retrievel dan Input Pertanyaan ..... | 14 |
| Kode Program 4. 4 Output Jawaban .....                        | 16 |

# **BAB I**

## **RETRIEVAL-AUGMENTED GENERATION**

### **1.1 Apa Itu Retrieval-Augmented Generation**

Retrieval-Augmented Generation (RAG) adalah sebuah kerangka kerja (framework) dalam kecerdasan buatan, khususnya pada pemrosesan bahasa alami (NLP), yang menggabungkan kemampuan model bahasa besar (Large Language Models / LLM) dengan sistem pencarian informasi (information retrieval) dari basis pengetahuan eksternal. Konsep RAG pertama kali diperkenalkan secara komprehensif oleh Lewis et al. (2020) untuk menyelesaikan tugas-tugas NLP yang padat pengetahuan (knowledge-intensive tasks).

Secara mendasar, LLM memiliki keterbatasan karena pengetahuannya bersifat statis (terbatas pada data latihannya saja) dan rentan mengalami halusinasi (hallucination), yaitu menghasilkan informasi yang terdengar masuk akal namun secara faktual salah (Procko & Ochoa, 2024). RAG hadir untuk mengatasi kelemahan tersebut dengan cara mencari (retrieval) informasi atau dokumen yang relevan dari basis data eksternal yang terpercaya, sebelum model bahasa (generator) menghasilkan jawaban.

Dalam prosesnya, dokumen-dokumen yang ada di basis pengetahuan eksternal tidak dimasukkan secara utuh, melainkan dipecah menjadi bagian-bagian yang lebih kecil yang disebut sebagai "chunk". Chunking ini diperlukan agar sistem pencarian dapat menemukan potongan informasi yang paling spesifik dan relevan dengan pertanyaan pengguna, yang kemudian digabungkan sebagai konteks tambahan bagi LLM untuk menyusun jawaban yang akurat, relevan, dan faktual (Mao et al., 2025).

## **BAB II**

### **METODE Pencarian Informasi pada RAG**

Dalam arsitektur Retrieval-Augmented Generation (RAG), fase retrieval (pencarian) adalah komponen krusial yang menentukan kualitas konteks yang akan diberikan kepada Large Language Model (LLM). Kinerja LLM sangat bergantung pada seberapa relevan dokumen atau chunk yang berhasil ditarik dari basis data. Secara umum, terdapat dua pendekatan utama dalam sistem temu kembali informasi (information retrieval), yaitu Lexical Retrieval dan Semantic Retrieval.

#### **2.1 Lexical Retrieval**

Lexical Retrieval atau pencarian berbasis kata kunci adalah metode tradisional dalam temu kembali informasi yang beroperasi dengan cara mencocokkan kemunculan kata demi kata (exact keyword matching) antara pertanyaan pengguna (query) dengan dokumen yang ada di dalam basis data. Metode ini merepresentasikan teks dalam bentuk vektor jarang (sparse vectors), di mana setiap dimensi vektor mewakili satu kata dalam kosakata (Robertson & Zaragoza, 2009).

Algoritma yang paling populer dan menjadi standar industri untuk lexical retrieval adalah BM25 (Best Matching 25) yang merupakan pengembangan dari metode TF-IDF (Term Frequency-Inverse Document Frequency). BM25 memberikan bobot pada dokumen berdasarkan seberapa sering kata kunci dari query muncul di dalam dokumen tersebut, sambil menyeimbangkannya dengan panjang dokumen agar dokumen yang panjang tidak mendominasi hasil pencarian (Robertson & Zaragoza, 2009).

Keunggulan utama dari pencarian leksikal adalah kecepatan komputasinya yang tinggi dan keakuratannya dalam menemukan entitas spesifik, seperti nama orang, singkatan, nomor ID, atau istilah teknis. Namun, metode ini memiliki kelemahan fatal yang dikenal sebagai masalah ketidakcocokan kosakata (vocabulary mismatch). Jika pengguna menggunakan kata bersinonim (misal: query menggunakan kata "mobil" sedangkan dokumen berisi kata "kendaraan roda empat"), sistem leksikal akan gagal mengidentifikasi relevansi di antara keduanya karena secara harfiah karakternya berbeda.

#### **2.2 Semantic Retrieval**

Untuk mengatasi keterbatasan lexical retrieval, diperkenalkanlah Semantic Retrieval. Metode ini tidak lagi mencari kecocokan karakter secara harfiah, melainkan mencoba memahami "makna" atau niat (intent) di balik query pengguna. Dalam pencarian semantik, baik query maupun chunk dokumen diproses menggunakan model jaringan saraf tiruan (neural

networks), seperti BERT, untuk diubah menjadi representasi matematika berupa vektor padat bernilai kontinu (dense vectors atau embeddings) dalam ruang dimensi tinggi (Reimers & Gurevych, 2019).

Dalam ruang vektor tersebut, teks yang memiliki makna semantik yang mirip akan berada pada posisi yang berdekatan, meskipun tidak ada satupun kata yang sama persis di antara keduanya. Untuk mencari dokumen yang relevan, sistem akan menghitung jarak metrik (misalnya Cosine Similarity atau Dot Product) antara titik vektor query dengan titik-titik vektor dokumen.

Salah satu model semantic retrieval yang paling mendasari implementasi RAG modern adalah DPR (Dense Passage Retrieval) yang diperkenalkan oleh Karpukhin et al. (2020). Model ini membuktikan bahwa pencarian berbasis embedding dapat mengungguli BM25 secara signifikan dalam tugas tanya-jawab (Question Answering) karena mampu memahami konteks dan menangani sinonim serta parafrase dengan sangat baik.

### **2.3 Perbedaan Lexical Retrieval dan Semantic Retrieval**

Berdasarkan penjelasan di atas, perbedaan antara semantic dan lexical retrieval dapat diringkas dalam beberapa aspek kunci berikut:

1. Mekanisme Pencocokan (Matching Mechanism): Lexical retrieval bertumpu pada pencocokan huruf/string yang persis sama (exact match), sedangkan semantic retrieval bertumpu pada kedekatan makna dan pemahaman konteks bahasa.
2. Representasi Data: Lexical retrieval menggunakan algoritma statistik (seperti BM25) yang menghasilkan vektor jarang (sparse vector) dengan memori yang efisien. Sebaliknya, semantic retrieval menggunakan model Deep Learning yang menghasilkan vektor padat (dense vector/embeddings).
3. Kelebihan Spesifik: Leksikal sangat unggul dalam mencari kata kunci spesifik (seperti error code komputer atau nama spesifik yang jarang). Semantik sangat unggul untuk query berbentuk kalimat tanya kompleks, penanganan sinonim, dan bahasa natural.
4. Kebutuhan Komputasi: Pencarian leksikal sangat ringan dan tidak memerlukan unit pemrosesan grafis (GPU). Pencarian semantik membutuhkan daya komputasi yang lebih besar (seringkali GPU) untuk melakukan encoding teks menjadi vektor pada saat indeksasi dokumen maupun pencarian.

Dalam arsitektur RAG tingkat lanjut saat ini, praktisi sering kali tidak memilih salah satu, melainkan menggabungkan keduanya menjadi Hybrid Search. Hybrid Search menggabungkan skor dari BM25 dan Dense Retrieval untuk mendapatkan hasil yang

mengerti makna (semantik) namun tidak kehilangan sensitivitas terhadap kata kunci spesifik leksikal (Procko & Ochoa, 2024).



## **BAB III**

### **ARSITEKTUR RAG, MANFAAT, DAN PERBANDINGANNYA DENGAN FINE-TUNING**

#### **3.1 Arsitektur Dasar RAG**

Secara operasional, kerangka kerja Retrieval-Augmented Generation (RAG) modern beroperasi melalui tiga tahapan utama atau pipeline arsitektur (Procko & Ochoa, 2024). Ketiga tahapan tersebut adalah:

1. **Pengindeksan (Indexing):** Ini adalah tahap persiapan data eksternal (basis pengetahuan). Dokumen-dokumen mentah (PDF, teks, database) dibersihkan dan dipecah menjadi potongan-potongan kecil yang disebut *chunks*. Setiap *chunk* kemudian dikonversi menjadi vektor representasi matematis (*embeddings*) menggunakan model encoder, lalu disimpan di dalam Basis Data Vektor (*Vector Database*).
2. **Pencarian (Retrieval):** Ketika sistem menerima pertanyaan (*query*) dari pengguna, *query* tersebut diubah menjadi *embedding* menggunakan model encoder yang sama dengan saat tahap pengindeksan. Sistem lalu mencari dan mengambil (*retrieval*) beberapa *chunk* teratas (*Top-K*) dari basis data vektor yang memiliki jarak matematis terdekat atau kemiripan tertinggi dengan *query* pengguna.
3. **Pembangkitan (Generation):** Dokumen atau *chunk* relevan yang telah diambil kemudian digabungkan dengan pertanyaan awal pengguna (membentuk *augmented prompt*). Konteks gabungan ini kemudian diumpankan (*fed*) ke Large Language Model (LLM), yang akan membaca konteks tersebut untuk menyusun dan membangkitkan jawaban akhir yang natural dan faktual.

#### **3.2 Manfaat RAG**

Implementasi arsitektur RAG pada model bahasa membawa sejumlah manfaat signifikan dibandingkan menggunakan LLM murni:

1. **Mitigasi Halusinasi:** LLM memiliki kecenderungan untuk memalsukan fakta (*halusinasi*) ketika mereka tidak tahu jawabannya. RAG secara drastis mengurangi hal ini dengan memaksa model untuk membatasi jawabannya hanya pada konteks referensi yang diberikan.
2. **Pengetahuan yang Selalu Mutakhir (Up-to-Date):** RAG memungkinkan LLM untuk menjawab isu-isu terbaru (seperti berita hari ini atau data perusahaan terkini) tanpa

perlu melatih ulang model. Cukup dengan memperbarui isi basis data vektor, pengetahuan sistem secara otomatis ikut terbaru (Lewis et al., 2020).

3. **Transparansi dan Keterlacakan (Traceability):** Karena jawaban dihasilkan dari dokumen referensi yang ditarik secara eksplisit, sistem RAG dapat memberikan rujukan (citations) yang memperlihatkan dari paragraf atau dokumen mana jawaban tersebut berasal, meningkatkan kepercayaan pengguna.

### 3.3 Perbedaan RAG Dengan Fine-Tuning

Dalam upaya menyesuaikan LLM dengan data spesifik perusahaan atau domain tertentu, para peneliti dan praktisi sering dihadapkan pada dua pilihan utama: RAG atau *Fine-Tuning*. Keduanya adalah pendekatan yang berbeda secara fundamental.

Menurut Balaguer et al. (2024), *Fine-Tuning* adalah proses memperbarui parameter internal (bobot saraf) dari sebuah model bahasa dengan melatihnya ulang menggunakan kumpulan data spesifik (*dataset*). Sementara itu, RAG membiarkan model bahasa tetap utuh (tanpa mengubah bobot internalnya) dan hanya menyuntikkan data eksternal pada saat inferensi melalui *prompt*.

Perbedaan utama antara keduanya dapat dianalogikan sebagai berikut: *Fine-Tuning* itu ibarat seorang siswa yang "belajar dan menghafal" materi secara intensif sebelum ujian. Pengetahuannya melekat pada otaknya, tetapi sangat sulit jika harus disuruh melupakan informasi yang salah atau memperbarui hafalan secara instan. Sebaliknya, RAG ibarat siswa yang mengikuti ujian "buka buku" (*open-book exam*). Siswa tersebut (LLM) tidak perlu menghafal semua isi perpustakaan, ia hanya butuh kemampuan membaca, karena ia dibekali alat pencari untuk membuka buku referensi yang relevan (Balaguer et al., 2024). Perbedaan Konseptual antara RAG dengan *Fine-Tuning* sebagai berikut:

1. **Fokus Penggunaan:** *Fine-Tuning* lebih optimal dan disarankan untuk mengajarkan model perilaku, gaya bahasa (*style*), format khusus, atau keahlian spesifik (misalnya memprogram bahasa tertentu). Sebaliknya, RAG jauh lebih unggul untuk mengajarkan fakta, pengetahuan domain, dan informasi baru.
2. **Pembaruan Data:** Memperbarui data pada model yang di-*fine-tune* memerlukan pelatihan ulang (*retraining*) yang memakan waktu dan biaya mahal. Pada RAG, pembaruan data sangat dinamis dan instan; cukup dengan menambah atau menghapus dokumen dari *database*.
3. **Biaya Komputasi:** *Fine-tuning* membutuhkan daya komputasi tinggi (GPU khusus pelatihan) dan biaya rekayasa data yang besar. RAG berbiaya jauh lebih murah karena

hanya bergantung pada komputasi pencarian (*retrieval search*) dan inferensi normal LLM (Balaguer et al., 2024).

Banyak studi modern kini merekomendasikan penggunaan kombinasi keduanya (misalnya *fine-tuning* model dasar agar lebih patuh pada instruksi format, dan RAG untuk injeksi pengetahuannya) agar mendapatkan hasil yang paling optimal.

## BAB IV

### PEMBAHASAN DAN UJI COBA

#### 4.1 Uji Coba Program

Pada uji coba kali ini kami membuat program sederhana menggunakan kode pemrograman python di google collab. Berikut kode program beserta penjelasannya.

```
import os
import chromadb
import google.generativeai as genai
from google.colab import userdata

# Setup Gemini API Key for the Generation stage
api_key = userdata.get('GEMINI_API_KEY')
genai.configure(api_key=api_key)

chroma_client = chromadb.PersistentClient(path="./chroma_storage")

# 3. Create or get a "Collection" (like a table in a database)
try:
    collection = chroma_client.get_collection(name="tugas_stki")
    chroma_client.delete_collection(name="tugas_stki")
    collection = chroma_client.create_collection(name="tugas_stki")
except:
    collection = chroma_client.create_collection(name="tugas_stki")

print("Local ChromaDB successfully initialized!")
```

**Kode Program 4. 1 Inisialisasi API dan ChromeDB**

Kode tersebut merupakan tahap inisialisasi awal yang berfungsi untuk mempersiapkan model bahasa AI dan membangun pondasi basis data penyimpanan sebelum sistem RAG dijalankan. Pada bagian pertama, kode memanggil pustaka-pustaka yang diperlukan dan secara aman mengambil kunci API Gemini melalui fitur rahasia Google Colab (userdata) untuk mengonfigurasi akses ke model kecerdasan buatan Google. Selanjutnya, program membuat sebuah basis data vektor menggunakan ChromaDB yang diatur menjadi persisten, artinya seluruh data yang dimasukkan nanti akan disimpan secara fisik di dalam folder lokal ./chroma\_storage sehingga tidak akan hilang meskipun sesi program diulang. Inti dari blok kode ini ada pada penggunaan logika try-except saat menyiapkan "koleksi" (yang bisa diibaratkan seperti tabel dalam sebuah basis data) dengan nama tugas\_stki. Logika ini dirancang sebagai fitur pembersih otomatis; program akan mengecek apakah koleksi tersebut sudah ada, lalu menghapusnya dan membuat yang baru dari nol agar data lama tidak

menumpuk atau menjadi ganda ketika kamu menjalankan sel tersebut berkali-kali. Jika koleksi tersebut memang belum pernah ada sama sekali, program akan langsung melompat ke blok `except` untuk membuatkan koleksi baru yang masih bersih dan siap digunakan.

```
# Our internal documents/knowledge
internal_documents = [
    "Mom has been cooking fried rice in the kitchen since 6 AM.",
    "My younger sibling usually goes to their room to sleep at 9 PM.",
    "Dad goes to work using a black motorcycle.",
    "In office dad is fishing",
    "My older sibling is currently studying Computer Science in the 4th semester.",
    "Our pet cat is named Meong and he likes to eat tuna fish."
]

ids = [f"id{i}" for i in range(len(internal_documents))]

# Save raw documents to ChromaDB
collection.add(
    documents=internal_documents,
    ids=ids
)

print(f"Successfully saved {collection.count()} documents to local ChromaDB!")
```

#### Kode Program 4. 2 Memasukan Document

Blok kode ini berfungsi sebagai tahap *Data Ingestion* atau proses memasukkan data ke dalam basis pengetahuan sistem RAG. Pertama, program mendefinisikan variabel `internal_documents` yang berisi sekumpulan kalimat fakta internal, yang nantinya akan menjadi sumber referensi bagi AI. Karena sistem basis data ChromaDB mewajibkan setiap entri data memiliki pengenalan yang spesifik, program kemudian membuat daftar ID unik secara otomatis menggunakan iterasi panjang dokumen, sehingga menghasilkan penomoran seperti 'id0', 'id1', dan seterusnya. Setelah data teks dan ID siap, perintah `collection.add` dipanggil untuk menyuntikkan data tersebut ke dalam koleksi database lokal yang telah dibuat pada tahap inisialisasi. Sebagai penutup, program melakukan verifikasi keberhasilan dengan mencetak jumlah total dokumen yang kini tersimpan di dalam ChromaDB menggunakan fungsi `collection.count()`.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# User query input
user_query = "What does dad do?"
print(f"User Query: '{user_query}'\n")

all_db_data = collection.get()
document_list = all_db_data['documents']

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(document_list)
query_vector = vectorizer.transform([user_query])

similarity_scores = cosine_similarity(query_vector,
tfidf_matrix).flatten()

top_k = 3
top_indices = np.argsort(similarity_scores)[::-1][:top_k]

print(f"=== RETRIEVAL RESULT (TOP-{top_k}) ===")
selected_contexts = []
for i in top_indices:
    doc = document_list[i]
    score = similarity_scores[i]

    if score > 0:
        selected_contexts.append(doc)
        print(f"-> Score: {score:.4f} | Document: '{doc}'")

gabungan_konteks = "\n".join(selected_contexts)

if not gabungan_konteks:
    gabungan_konteks = "No relevant context found."
    print("-> No relevant context found.")

```

#### Kode Program 4. 3 Proses Retrieval dan Input Pertanyaan

Blok kode ini merupakan fase *Retrieval* (pencarian konteks) dalam arsitektur RAG yang bertugas mengekstraksi informasi paling relevan dari basis data sebelum diproses oleh model LLM. Proses dimulai dengan mengunduh seluruh dokumen dari ChromaDB, kemudian teks-teks tersebut bersama dengan pertanyaan pengguna diubah menjadi representasi vektor matematis menggunakan metode TfidfVectorizer. Representasi vektor ini memungkinkan komputer mengukur tingkat kemiripan tekstual antara pertanyaan dan setiap dokumen menggunakan perhitungan matriks cosine\_similarity, yang menghasilkan rentang nilai skor kedekatan. Program kemudian mengurutkan skor tersebut secara menurun (descending) menggunakan fungsi np.argsort() untuk mengekstrak tiga dokumen teratas (Top-K = 3) yang

paling relevan. Sebagai tahapan filtrasi akhir, program hanya memilih dokumen yang memiliki skor di atas nol, lalu menggabungkannya menjadi satu teks konteks yang utuh (gabungan\_konteks), sehingga model AI pada tahap selanjutnya memiliki landasan fakta yang kuat untuk merumuskan jawaban akhir.

```
import requests
import json
import time

# CONSTRUCTING A STRICT PROMPT
rag_prompt = f"""
You are a very intelligent, concise, and to-the-point assistant.
Your task is to answer user questions ONLY based on the context provided
below.

STRICT RULES:
1. Answer directly to the core of the question, concisely, precisely, and
naturally (to the point).
2. DO NOT repeat or copy entire sentences from the context verbatim.
3. DO NOT include introductory phrases like "Based on the context..." or
"According to the text...".
4. If the answer is not contained in the context, reply with "I don't know
based on the provided context."

Context:
{gabungan_konteks}

Question: {user_query}
Concise Answer:
"""

print("Sending contexts from ChromaDB to Gemini-2.5-Flash (Concise
Mode)...")

url = f"https://generativelanguage.googleapis.com/v1/models/gemini-2.5-
flash:generateContent?key={api_key}"
headers = {'Content-Type': 'application/json'}
payload = {
    "contents": [{
        "parts": [{"text": rag_prompt}]
    }]
}

max_retries = 5
delay = 1
api_success = False

for attempt in range(max_retries):
    response = requests.post(url, headers=headers,
data=json.dumps(payload))
```

```

res_json = response.json()

if response.status_code == 200:
    api_success = True
    break

error_code = res_json.get('error', {}).get('code')

if error_code in [503, 429]:
    print(f"Attempt {attempt + 1} failed (HTTP {error_code}). Server
sibuk. Mencoba lagi dalam {delay} detik...")
    time.sleep(delay)
    delay *= 2
else:
    raise Exception(f"Gemini API Error: {res_json}")

if not api_success:
    raise Exception(f"Gemini API Error: Gagal setelah {max_retries}
percobaan.")

final_answer = res_json['candidates'][0]['content']['parts'][0]['text']

print("\n=== RAG PROGRAM FINAL ANSWER ===")
print(final_answer.strip())

```

#### Kode Program 4.4 Output Jawaban

Blok kode terakhir ini merupakan fase *Generation* (pembuatan jawaban) yang bertugas memproses informasi hasil pencarian menjadi jawaban akhir yang natural dan akurat menggunakan model *Large Language Model* (LLM) Gemini. Proses dimulai dengan teknik *Prompt Engineering*, di mana sistem secara dinamis menyusun sebuah instruksi komprehensif (rag\_prompt). Instruksi ini tidak hanya menggabungkan pertanyaan pengguna dan teks konteks yang relevan (gabungan\_konteks), tetapi juga menyertakan aturan ketat (*strict rules*) agar AI menjawab secara langsung, ringkas, dan tidak mengarang informasi di luar konteks (mencegah halusinasi). Selanjutnya, *prompt* tersebut dibungkus dalam format JSON dan dikirimkan ke *endpoint* API Gemini melalui *HTTP POST request* yang dimana diakhir akan diberikan jawaban dari pertanyaan sebelumnya.

## 4.2 Pembahasan Hasil dan Analisis Uji Coba

Berdasarkan pengujian sistem *Retrieval-Augmented Generation* (RAG) yang telah dilakukan, program menunjukkan hasil keluaran yang dibagi menjadi dua fase utama: fase *Retrieval* (pencarian dokumen) dan fase *Generation* (pembuatan jawaban). Berikut adalah analisis dari masing-masing fase berdasarkan *output* yang dihasilkan.



```
*** User Query: 'What does dad do?'

=== RETRIEVAL RESULT (TOP-3) ===
-> Score: 0.4303 | Document: 'In office dad is fishing'
-> Score: 0.3398 | Document: 'Dad goes to work using a black motorcycle.'
```

Gambar 4. 1 Hasil Retrieval

Gambar tersebut menampilkan hasil keluaran (*output*) dari fase pencarian dokumen atau *retrieval* dalam sistem RAG berdasarkan pertanyaan pengguna mengenai aktivitas ayah. Sistem berhasil menyaring dua dokumen teratas yang paling relevan, di mana fakta tentang ayah memancing di kantor menempati peringkat pertama dengan skor kemiripan tertinggi sebesar 0.4303. Sementara itu, dokumen mengenai ayah mengendarai motor hitam berada di urutan kedua dengan skor 0.3398 karena dianggap memiliki tingkat relevansi yang sedikit lebih rendah terhadap inti pertanyaan tersebut.

```
** Sending contexts from ChromaDB to Gemini-2.5-Flash (Concise Mode)...

=== RAG PROGRAM FINAL ANSWER ===
Dad is fishing.
```

Gambar 4. 2 Output Jawaban

Gambar tersebut menunjukkan hasil keluaran akhir (*final answer*) dari fase *generation* pada sistem RAG setelah seluruh konteks relevan dari ChromaDB dikirimkan ke model LLM Gemini-2.5-Flash. Berdasarkan analisis konteks yang diterima, model berhasil merumuskan jawaban yang sangat ringkas dan langsung pada sasaran, yaitu "*Dad is fishing.*" Hasil ini membuktikan efektivitas penerapan *strict prompt engineering* (mode *concise*) yang berhasil memaksa AI memberikan jawaban yang padat tanpa menyertakan kalimat pengantar atau pengulangan teks yang tidak perlu.

#### 4.2.1 Analisis Fase *Retrieval* (Pencarian Konteks)

Pada proses *retrieval*, sistem berhasil menyaring kumpulan dokumen internal dan menampilkan dokumen *Top-K* yang paling relevan. Output menunjukkan dua dokumen yang berhasil lolos dengan nilai *Cosine Similarity* lebih dari 0:

- Dokumen 1: "*In office dad is fishing*" dengan skor 0.4303
- Dokumen 2: "*Dad goes to work using a black motorcycle.*" dengan skor 0.3398

Hasil ini membuktikan bahwa metode ekstraksi fitur TF-IDF (*Term Frequency-Inverse Document Frequency*) yang dikombinasikan dengan metrik *Cosine Similarity* berjalan dengan

baik. Dokumen pertama mendapatkan skor lebih tinggi karena matriks vektornya memiliki irisan kata dan pembobotan makna yang lebih dekat dengan inti pertanyaan (*query*). Selain itu, karena program telah dilengkapi dengan filter skor ( $\text{score} > 0$ ), dokumen-dokumen lain di dalam *database* (seperti aktivitas ibu atau adik) secara otomatis dibuang karena memiliki skor kemiripan 0. Hal ini memastikan bahwa LLM hanya akan menerima "bahan baku" yang benar-benar relevan.

#### 4.2.2 Analisis Fase *Generation* (Pemrosesan LLM dan Prompt Engineering)

Setelah dokumen diserahkan kepada model LLM (Gemini-2.5-Flash), sistem menghasilkan jawaban akhir yang sangat ringkas, yaitu: "Dad is fishing." Analisis terhadap keluaran ini menunjukkan keberhasilan penerapan teknik *Strict Prompt Engineering*. Terdapat dua faktor utama mengapa model merumuskan jawaban tersebut dan tidak menggabungkannya dengan dokumen kedua (tentang sepeda motor):

a) Pemahaman Semantik Secara Natural (NLP)

Model LLM tidak hanya mencocokkan kata secara kaku, tetapi juga memahami makna semantik di balik pertanyaan (misalnya, pertanyaan mengenai aktivitas/pekerjaan). Model secara cerdas mengidentifikasi bahwa "*fishing*" adalah inti aktivitas yang ditanyakan, sedangkan "*using a black motorcycle*" hanyalah informasi pelengkap mengenai metode transportasi.

b) Kepatuhan terhadap *Constraint* (Batasan) Prompt

Instruksi khusus yang disematkan pada kode, yakni "*Answer directly to the core of the question, concisely, precisely, and naturally (to the point)*", telah memaksa model untuk beroperasi dalam mode ringkas (*Concise Mode*). Model secara otomatis memangkas informasi ekstran (informasi tambahan yang tidak wajib) demi menghasilkan jawaban yang langsung pada sasaran (*to the point*).

## **BAB V**

### **KESIMPULAN**

#### **5.1 Kesimpulan**

Secara keseluruhan, uji coba ini mendemonstrasikan bahwa arsitektur RAG yang dibangun mampu memitigasi risiko *halusinasi* pada AI. Model tidak mengarang jawaban berdasarkan data latih publiknya, melainkan secara ketat mengekstraksi dan merangkum fakta hanya berdasarkan kumpulan basis pengetahuan lokal yang ditarik oleh ChromaDB. Kombinasi pencarian berbasis skor metrik (TF-IDF) dan inferensi kognitif (LLM) menghasilkan sistem tanya-jawab dokumen internal yang presisi dan efisien.

## DAFTAR PUSTAKA

- Balaguer, A., Benara, V., Cunha, R. L. de F., Filho, R. de M. E., Hendry, T., Holstein, D., Marsman, J., Mecklenburg, N., Malvar, S., Nunes, L. O., Padilha, R., Sharp, M., Silva, B., Sharma, S., Aski, V., & Chandra, R. (2024). *RAG vs Fine-tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture*. <http://arxiv.org/abs/2401.08406>
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. *EMNLP 2020 - 2020 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*, 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 2020-Decem*.
- Mao, Y., Ge, Y., Fan, Y., Xu, W., Mi, Y., Hu, Z., & Gao, Y. (2025). A survey on LoRA of large language models. *Frontiers of Computer Science*, 19(7), 1–144. <https://doi.org/10.1007/s11704-024-40663-9>
- Procko, T. T., & Ochoa, O. (2024). Graph Retrieval-Augmented Generation for Large Language Models: A Survey. *Proceedings - 2024 Conference on AI, Science, Engineering, and Technology, AIXSET 2024*, 166–169. <https://doi.org/10.1109/AIXSET62544.2024.00030>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *EMNLP-IJCNLP 2019 - 2019 Conference on Empirical Methods in Natural Language Processing and 9th International Joint Conference on Natural Language Processing, Proceedings of the Conference*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. In *Foundations and Trends in Information Retrieval* (Vol. 3, Number 4). <https://doi.org/10.1561/15000000019>

## LAMPIRAN

Link Kode Program

[https://colab.research.google.com/drive/14CUTPHtjR\\_oT3Csc\\_hFbhbUMDdytm6t5?usp=sharing](https://colab.research.google.com/drive/14CUTPHtjR_oT3Csc_hFbhbUMDdytm6t5?usp=sharing)